Checkpoint Report – Week 2

This report summarizes the progress completed during Week 2. The focus of this week was dataset preparation, preprocessing, CNN training, reinforcement learning threshold design, evaluation, and UI integration.

Dataset Preparation and Preprocessing

The Google Speech Commands V2 dataset was used as the primary source of audio data.

The dataset was organized into training, validation, and test splits. Audio files were converted into Mel-spectrogram representations using Short-Time Fourier Transform (STFT) to make them compatible with the convolutional neural network.

Data preprocessing scripts were implemented to load audio files, pad or trim signals, generate spectrograms, assign labels, and save processed tensors. The dataset pipeline was verified using test runs before model training. Additionally, exploratory data analysis was performed using a notebook to inspect class distribution and audio samples.

Model Training

A spectrogram-based convolutional neural network was implemented to classify speech commands. The model was trained using the processed dataset and evaluated on the test split. The CNN outputs probability scores for each command class: play, pause, next, stop, and unknown/other words. Additionally, training scripts and evaluation scripts were completed and tested.

Initial evaluation was down and results mainly indicate that the model can correctly classify most commands but still has difficulty with certain classes such as silence.

Evaluation

Initial evaluation was done and results mainly indicate that the model can correctly classify most commands but still has difficulty with certain classes such as silence.Initial results show about 0.83 accuracy, 0.72 Macro-F1, and expected cost around 0.41. Additionally, confusion matrix and prediction logs were saved.

Reinforcement Learning

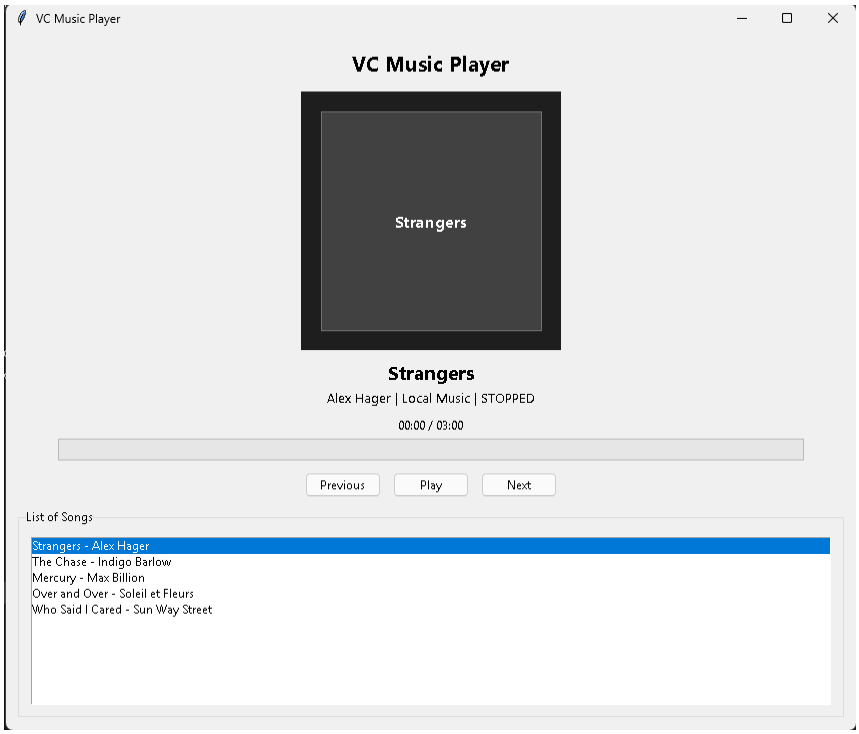
A reinforcement learning agent was stubbed to optimize the decision threshold of the classifier. Instead of always accepting the highest probability output, the system uses a threshold to decide whether the command should be executed. The reward function penalizes false activations more than missed commands to reduce unintended playback.

Natural Language Processing

A basic command-processing module was implemented to prepare for future NLP integration. This module will allow commands such as: playing song, stopping song, playing specific song, playing previous song, and playing next song.

User Interface

A simple user interface was implemented using Tkinter to demonstrate real-time interaction with the model. The UI allows real-time user interaction, processing of commands, running predictions, displaying detected commands, and navigating through the available song or playlist hands-free.



Documentation

The model card was created to describe the dataset source, model architecture, metrics, limitations, and ethical considerations. Additionally, the ethics statement was updated and discusses privacy risks from microphone input, accent bias in speech datasets, and false command activation. While the mitigation strategies include local processing and threshold optimization. These documentation files can be found in *docs/*.

Conclusion

In conclusion, the Week 2 milestones were successfully completed. The dataset was processed, the CNN model was trained, evaluation metrics were generated, and a prototype UI was created. A reinforcement learning threshold optimizer was stubbed, and documentation including the Model Card and ethics statement was updated.